

CCG Review Report

Independent devil's-advocate review · specialized-agent assisted

2026-09-18

- [Executive summary](#)
- [Method & scope](#)
- [Findings register](#)
 - [CRITICAL](#)
 - [HIGH](#)
 - [MEDIUM](#)
 - [LOW](#)
- [Rejected findings \(verified false — recorded for honesty\)](#)
- [What was genuinely done well \(not padding — calibration\)](#)
- [Disposition](#)
- [Appendix A — Architecture diagrams](#)
 - [A.1 Architecture context](#)
 - [A.2 Trust boundaries & threats](#)
 - [A.3 Deploy pipeline & data flow](#)

Executive summary

This is a **no-bluff** review of the Continuous Compliance Guardian (CCG) POC, produced by four specialized review agents (Python code, Terraform/IaC security, architecture/documentation, and a behavioral semantic reviewer over the full git history) with every high-severity finding independently verified against authoritative sources before being reported here.

Overall verdict: NEEDS CHANGES before any enforcement-mode deployment.

The offline domain model is genuinely well-built — the authorization ordering, server-side target resolution, fail-closed activation service, and redacting audit sink are coherent and tested. **The problem is that the safety guarantees are proven only against test doubles, while the production path and the one deployable Terraform configuration do not yet enforce them.** Two findings are CRITICAL because they invert or nullify the project's central security claim ("every dangerous action is authorized at an external Cedar boundary").

The single most important correction: **the codebase and its own docs describe the AgentCore LOG_ONLY engine mode as "fail-closed / forbids still enforce." That is factually wrong.** AWS's own service model states LOG_ONLY "evaluates... but does not enforce the decision." This was asserted as fact across the threat model, deploy runbook, and Terraform comments for multiple sessions and was missed until this review.

Severity	Count
Critical	2

Severity	Count
High	4
Medium	9
Low	5

Nothing is deployed to the cloud (sandbox `...8441` is a clean slate), so **none of these findings represent live exposure today**. They are gating issues that must be resolved before the enforcement-mode deploy — which is exactly the right time to catch them.

Method & scope

- **Agents used:** 3× context-gatherer (code / IaC / architecture) + 1× semantic_reviewer, each briefed adversarially (“assume something is wrong”).
- **Verification:** every Critical/High was re-checked by direct command against source of truth — the AWS `bedrock-agentcore-control` service model for LOG_ONLY semantics, `grep` for dead code, `pytest --collect-only` for the true test count, and `gh run list` to disprove one false-alarm finding.
- **Scope:** `src/ccg/` (16 modules), `tests/` (117 tests), all Terraform (`main.tf`, 5 modules), `.github/workflows/ci.yml`, `console/`, and all `docs/` + `.kiro/specs`.
- **Honesty note:** one agent finding (CI `-backend=false` conflicting with the `cloud{}` block) was **not reproduced** — the terraform CI job is green on every recent merge. It is listed under “Rejected findings,” not as a defect.

Findings register

CRITICAL

C1 — LOG_ONLY engine mode is fail-OPEN, not fail-closed (the core safety claim is inverted)

- **Evidence:** `modules/agentcore-gateway/main.tf` binds `policy_engine_configuration { mode = "LOG_ONLY" }` with the comment “always-on forbids still evaluate, but no permit is enforced — the fail-closed posture.” `docs/THREAT_MODEL.md` and `docs/DEPLOY_RUNBOOK.md` repeat this.
- **Verified fact (AWS `bedrock-agentcore-control` service model, `GatewayPolicyEngineConfiguration.mode`):** “LOG_ONLY — The policy engine evaluates each action against your policies and adds traces on whether tool calls would be allowed or denied, **but does not enforce the decision.**” Only `ENFORCE` allows/denies.
- **Impact:** In LOG_ONLY, **forbids are logged, not blocked**. If the Gateway is ever enabled with any permit present, a mutation Cedar would deny (including a `prod` target or `CloudTrail-disable`) is allowed through. The schema-capture deploy is safe **only because zero permits exist** — not because forbids enforce. The design invariant “the Gateway enforces deny for calls with no active permit” is not met by anything deployable today.

- **Recommendation:** Stop describing LOG_ONLY as fail-closed everywhere. Document that capture safety rests on *no permits existing*. Enforcement requires `mode = "ENFORCE"` with a default-deny Cedar policy set, activated via the audited flow after T2 is resolved. Keep `enable_agentcore_gateway = false` until then.

C2 — Production gateway path enforces nothing; all safety lives in the test double

- **Evidence:** `src/ccg/gateway.py` — `AgentCoreGatewayClient.invoke` is `return self._transport.invoke(request)`. No decision eval, no default-deny short-circuit, no idempotency, no audit. Every one of those behaviors exists **only** in `src/ccg/testing.py::InMemoryCedarGateway`. `gateway.py` has **zero tests**.
- **Impact:** Every test proving “forbid-overrides-permit / default-deny / idempotent / single-execution” runs against a fake that will not exist in production. The `deny → activate → allow` demo works only because `demo.py` manually calls `gateway.activate_permits(...)` — **no production code propagates a control-plane activation to the enforcement point**. CCG-REQ-008/010/011 have no evidence against real AgentCore.
- **Recommendation:** Treat the Cedar-first guarantee as UNVALIDATED against real infrastructure. Add an integration seam that runs the same authz assertions against the real transport before any permit activation. Share a common invoke skeleton so the double and prod cannot diverge. Do not cite the offline tests as proof of the deployed boundary.

HIGH

H1 — IAM tool-execution tag conditions fail-open or dead on tag-unaware actions

- **Evidence:** `modules/iam-policies/main.tf` — `S3EncryptionEnablement` allows `s3:PutEncryptionConfiguration` on `arn:aws:s3:::*` gated by `aws:ResourceTag/CCGDemo`; S3 bucket ops do **not** honor `aws:ResourceTag`. Same class of issue for `tag:GetResources`. The module comment claims “no bare `*` resource except read-only.”
- **Impact:** The tag gate silently doesn’t apply — the statement is either dead (remediation never works) or broader than “sandbox-tagged only.” The “least-privilege / sandbox-scoped” claim is false for these statements.
- **Recommendation:** Scope S3 by explicit bucket ARNs, not `arn:aws:s3:::*` + an unenforceable tag condition. Verify each action against the IAM service-authorization reference “condition keys” column before claiming tag-gating. Split read (`tag:GetResources`, unscopable) from write.

H2 — `observability.py` is dead code; the threat model’s detection layer never fires

- **Evidence:** `grep` across `src/ccg` confirms `Metric / MetricSink / .increment( / .gauge(` appear only inside `observability.py` and `tests/test_observability.py`. No production module emits a metric.
- **Impact:** `docs/THREAT_MODEL.md` maps all eight threats (T1–T8) to a `Metric` alarm as their runtime detection control. **None of those alarms can fire** — the metrics are never incremented on a deny, audit failure, or partial run. `TEST_TRACEABILITY.md` cites `test_observability` as CCG-REQ-037 coverage, but that test only checks enum arithmetic.
- **Recommendation:** Inject a `MetricSink` through `discovery/gateway/policy/reconcile/audit` and emit at each decision/outcome, with emission assertions in tests — OR downgrade every

threat-model “Metric:” line and the CCG-REQ-037 row to “planned (gated on 6.3).” Do not present unwired metrics as controls.

H3 — Idempotency exists only in the double, is not payload-bound, and caches failures as terminal

- **Evidence:** `src/ccg/testing.py` — replay keyed on `request.idempotency_key` alone (no hash of tool/target/params); the `except` branch stores a synthesized `FAILED` into the replay cache. `AgentCoreGatewayClient` has no replay map at all.
- **Impact:** (a) A reused key with different parameters returns the prior result and **silently skips the new mutation**. (b) A transient failure becomes a **permanent** replayed failure for that key, contradicting “uncertain result pauses for reconciliation.” (c) Production has no idempotency, so redelivery double-executes.
- **Recommendation:** Bind the idempotency record to a hash of `(tool, target_arn, sorted params, expected_finding_version)`; reject key reuse with a changed payload (conflict). Never cache non-terminal failures. Specify idempotency as a required behavior of the real transport and test it there.

H4 — `PolicyControlPlaneTransport` Protocol is missing `update_policy_set` — latent crash on the activation happy path

- **Evidence:** `src/ccg/policy.py` — the Protocol (≈:29-36) declares only `get_state / force_deny_safe`, but `PolicyActivationService.apply` calls `self._transport.update_policy_set(...)` (≈:88). The in-memory double happens to implement it; a spec-conformant production transport implementing only the Protocol will `AttributeError`.
- **Impact:** The first real activation would crash rather than activate — on the exact path that is supposed to be audited and fail-safe.
- **Recommendation:** Add `update_policy_set` to the Protocol. Make generation-based optimistic concurrency the single source of truth; drop or make durable the in-memory `request_id` cache (it doesn’t survive a process restart).

MEDIUM

- **M1 — Reconcile resolves on an unbound boolean.**
`reconcile.py::confirm_from_discovery(*, drift_still_present: bool)` has no discovery-run/correlation/evidence binding and no partial-run guard. A `False` from a throttled/partial read would RESOLVE a finding — the T8 vector the design forbids. *Fix: pass a discovery-run correlation and validate it came from a completed, non-partial run.*
- **M2 — T2 only half-closed: tool + parameters are caller-controlled and unvalidated against the finding.** `api.py` resolves the `target` server-side (good) but passes `command.tool` and `command.parameters` through unchecked. A remediator can point any registered tool at any finding, or supply arbitrary `desired_tags` /CIDR (non-global). *Fix: derive/validate the tool and a bounded parameter set from the finding’s `rule_id` server-side.*
- **M3 — “Frozen” contracts are shallow-copied; nested mutable values alias caller data.**
`contracts.py` does `dict(self.parameters)` etc.; a nested list/dict inside `parameters` /

- `attributes / details` remains shared with the caller and mutable post-construction. *Fix: deep-freeze (recursive tuple/ MappingProxyType) or document the invariant.*
- **M4 — DynamoDB ships without deletion protection by default.** `findings-store` `enable_deletion_protection` defaults `false` despite the “guard against accidental deletion” comment; PITR doesn’t survive a table delete. *Fix: default `true` ; make teardown flip it off explicitly.*
- **M5 — At-rest encryption uses AWS-owned keys, not KMS — no key policy, no CloudTrail key-usage trail.** DynamoDB SSE `enabled=true` with no `kms_key_arn` , and the S3 bucket is SSE-S3. Acceptable per the documented AWS-managed-keys cost decision, but it is a real Security-pillar gap for a *compliance* product and must be stated, not silently accepted. *Fix: document explicitly (ADR); revisit CMK for production.*
- **M6 — Audit bucket deny only catches a missing SSE header, not a wrong algorithm; no access logging.** `DenyUnencryptedObjectUploads` uses `Null` on the SSE header, so an upload with `aws:kms` under an arbitrary key passes. No `aws_s3_bucket_logging` on the audit bucket. *Fix: pin `AES256` via `StringNotEquals` deny; add access logging.*
- **M7 — Audit redaction misses value-borne secrets and over-redacts identifiers.** `audit.py::_redact` matches by key substring only; a secret string inside a list (no key) passes in cleartext, while `AccessKeyId` (a non-secret identifier operators need) is blanked because “accesskey” matches. *Fix: add value-pattern redaction (AKIA..., PEM, JWT); narrow the key match; carry parent-key context into list elements.*
- **M8 — CI pins a Terraform version that does not exist (1.15.8).** `setup-terraform` may fail to fetch or silently fall back, weakening the validate gate. *Fix: pin a real 1.x release consistent with `versions.tf` (>=1.8,<2.0).*
- **M9 — Documentation self-contradicts on test count and overclaims coverage.** Docs say both “22-test suite” (`CAPABILITY_BASELINE` , `DESIGN_REVIEW_RECONCILIATION`) and “108 tests” (`TEST_TRACEABILITY`); the true count is **117**. `TEST_TRACEABILITY` maps CCG-REQ-034 “structured logging” to tests, but **there is no logging subsystem** (`grep` finds no `logging / getLogger`). *Fix: regenerate counts from one source; correct the logging/alarms rows to “planned.”*

LOW

- **L1 — No ADRs exist.** Every load-bearing decision (Cedar-first, LOG_ONLY posture, AWS-managed keys, dry-run rotation, provider pin, Python 3.14, HCP OIDC) lives only in scattered prose. *Fix: `docs/adr/` — addressed in this change set.*
- **L2 — Console is wired to nothing and holds client-side policy knowledge.** `console/app/page.tsx` hardcodes `requestedVersion: "v1"` on activate — the exact “client must not hold policy knowledge” the design forbids; no console tests; decision-evidence not displayed. *Fix: drive version from server state; add tests; render decision evidence.*
- **L3 — README is stale/inaccurate.** 12 lines; doesn’t state “planning-stage POC, nothing deployed”; Quick Start says `terraform -chdir=terraform init` but there is no `terraform/` directory (root is the TF root). *Fix: rewrite — addressed in this change set.*
- **L4 — Gateway assume-role uses `aws:SourceAccount` only.** `aws:SourceArn` (the specific Gateway ARN) is stronger confused-deputy protection; the comment calling `SourceAccount` “confused-deputy protection (T1)” overstates it (account-level, not resource-level). *Fix: add `aws:SourceArn` once the Gateway ARN is known.*

- **L5 — Policy-JSON root outputs are not** `sensitive = true`. Not secrets, but they render the exact authorization surface in plaintext logs/state. *Fix:* `mark sensitive = true`.

Rejected findings (verified false — recorded for honesty)

- **CI** `terraform init -backend=false` **conflicts with the** `cloud{}` **block** → **CI broken**. *Not reproduced.* `gh run list` shows the `terraform` job `success` on every recent merge to `main`. Modern Terraform permits `-backend=false` validate with a cloud block. No action.

What was genuinely done well (not padding — calibration)

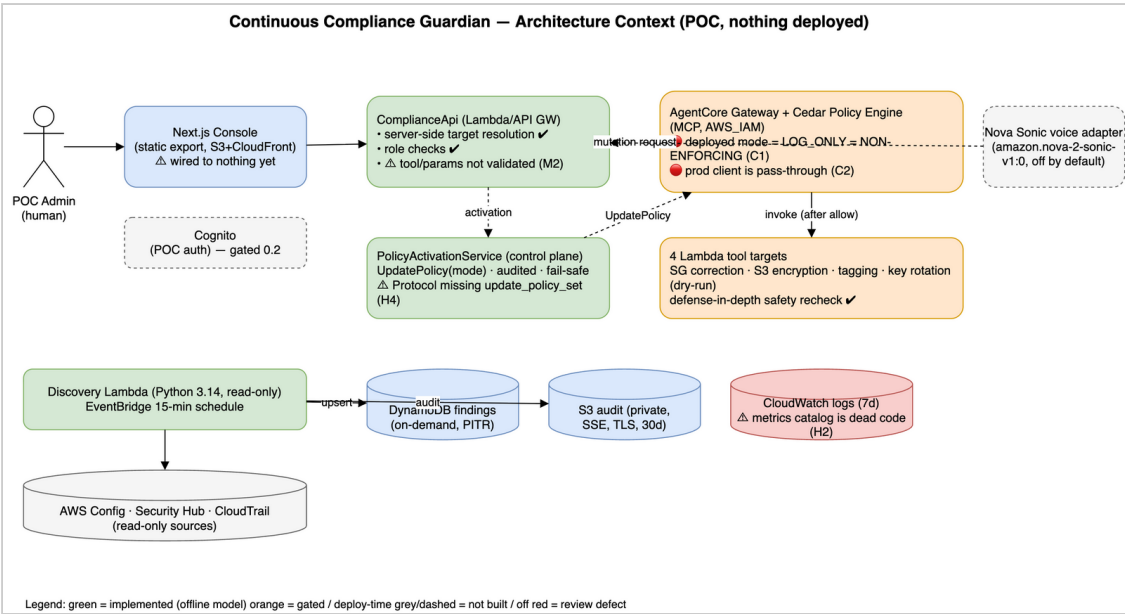
- `api.py` server-side **target** resolution truly closes target-injection (a command has no target field to inject).
- `authz.py` forbid-overrides-permit / default-deny ordering is correct and well-tested *as a model*.
- `policy.py` activation is audited and fail-safe-on-error (reconciles toward LOG_ONLY).
- The IAM action-name fix (PR #7) was a real catch — phantom `UpdateGatewayPolicy` → real `UpdatePolicy`.
- `discovery.py` / `sources.py` correctly treat partial/failed reads as non-compliant (T8 intent), and dedup is deterministic.
- Storage baseline (public-access block, ownership, versioning, lifecycle, TLS deny) is solid.

Disposition

Because nothing is deployed, the correct sequencing is: fix C1/C2 framing and the enforcement design **before** the enforcement-mode deploy; treat the schema-capture deploy (permits absent, LOG_ONLY) as still acceptable **with corrected documentation**. The prioritized remediation plan is in the companion document, `CCG_REMEDIATION_TODO`.

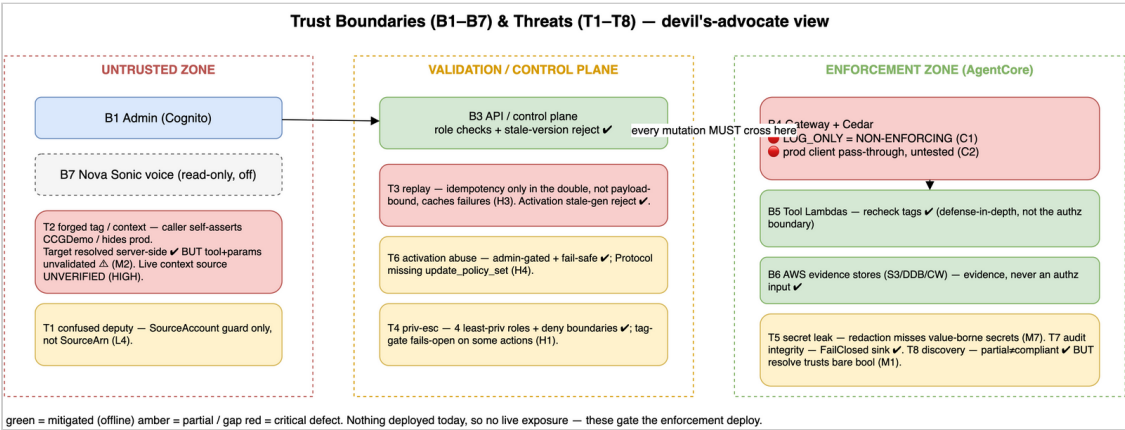
Appendix A — Architecture diagrams

A.1 Architecture context



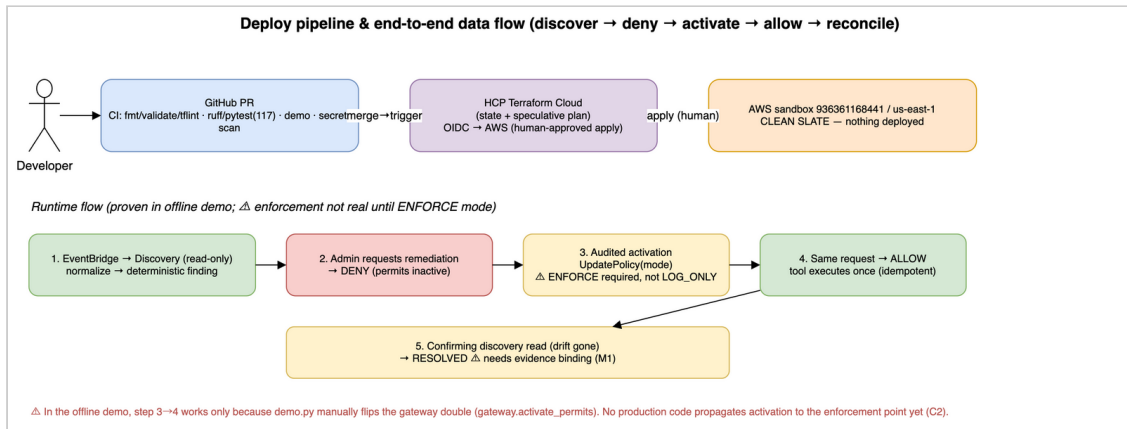
Architecture context

A.2 Trust boundaries & threats



Trust boundaries and threats

A.3 Deploy pipeline & data flow



Deploy and data flow